Rally team plan: features, sponsors and work split

Oct 24, 2026 · @Chaudhari kevin

Where Rally stands

Rally already runs end to end on both sides for three activities, so the remaining work is sponsor depth, a few standout features and demo polish, not new plumbing.

Works today (tested: 131 automated tests, plus a browser run as customer and vendor at once):

- Goal compiler with review page (rule-based; Gemini adapter written but never called against the real API)
- Hard-filter matching with rejection reasons, local ranking
- Group mediator negotiating one shared plan across 5 personal agents; scuba converges in 4 rounds
- Six vendors with provider agents, staff approval of quotes, booking confirmation, editable pricing
- Agent conversation log, 7-step workflow, group chat with voice input, read-aloud and translation
- Simulated commitment ledger with version-bound approvals

Still simulated or untested:

Piece	Today	Why it matters
Agents	In-process inside the API; real uAgents only in a separate local Bureau script	Fetch.ai judges will want registered agents talking over the network
Compiler	Rule-based templates	Gemini track needs real model calls
Ranking	Local TF-IDF	Moss is not connected
State	In-memory; Convex mirror never run against a deployment	Restart loses everything; Convex track needs it live

Piece	Today	Why it matters
Vendor prices	Fixture tables	Apify could pull real listed prices
Payments	Simulated ledger	x402 testnet not written
Hosting	Laptop only	Vultr track and judges' own phones need a URL

Sponsor alignment

Fetch.ai is the track Rally fits best, and its biggest gap is that judges must be able to use Rally from ASI:One chat, through agents registered on Agentverse. That one change is the top priority.

The Cal Hacks 13.0 rule pages were not reachable online, so the Fetch.ai row below uses the published rules from Fetch.ai's LA Hacks 2026 track as the best guide. Kevin confirms each sponsor's actual rules at the booths before 3 pm today.

Sponsor	What Rally does with it	What judges will see	Owner	Today
Fetch.ai (uAgents, Agentverse, ASI:One)	Personal, mediator and vendor agents registered on Agentverse; a Rally concierge agent with the Chat Protocol so a goal can be typed in ASI:One; optional Payment Protocol for a booking fee	An ASI:One chat that ends in a negotiated group plan; agent addresses and Innovation Lab badges in the README	Parikshit	Agents run in-process only

Sponsor	What Rally does with it	What judges will see	Owner	Today
Google Gemini	Goal compiler (structured output), plan explanations for each person, chat translation, vendor reply suggestions	The review page labelled "Compiled by Gemini"; a "Why this plan" card per person	Dev	Adapter written, never called
Moss	Semantic ranking of the intent board, and semantic search over vendor listings	Matches page labelled "Ranked by Moss" with similarity scores	Dev	Not connected
Convex	Live state for the group room and vendor portal; restart-safe	Two laptops update instantly with no polling; kill the API and the room survives	Kevin	Mirror code written, never deployed
Apify	An Actor that pulls real vendor listing prices into the vendor catalog, shown as "Listed price"	A third provider per activity sourced from the web, with its source link	Dev	Not started
Coinbase x402	Each member pays a small testnet deposit when committing	A Base Sepolia transaction link on the confirmation page	Kevin	Not started
Vultr	API and agents hosted on a Vultr VM; web on Vercel	A public URL judges open on their phones	Kevin	Laptop only
Visa	Unknown: ask the booth whether a Visa API is expected		Kevin	Unconfirmed

Fetch.ai's published judging weights: functionality 25%, use of Fetch.ai tech 20% (registered on Agentverse, Chat Protocol for ASI:One, a monetization approach), innovation 20%, real-world impact 20%, presentation 15%. Deliverables: ASI:One chat session URL, Agentverse agent URLs, public GitHub repo, 3 to 5 minute demo video (LA Hacks 2026 hackpack).

New features

Fifteen features, in priority order. P0 makes Rally eligible and credible for its main sponsors. P1 is what makes it memorable. P2 only starts after the 2 am feature freeze if time remains. Hours are estimates for one person.

#	Feature	Why it matters	Sponsor	Owner	Priority	Hours
1	Rally concierge on ASI:One: type a goal in ASI:One chat; the agent compiles it, finds the group, runs the negotiation and replies with the plan and a link to the group room	Fetch.ai requires the use case to work through ASI:One	Fetch.ai	Parikshit	P0	5
2	Real agent network: every personal, mediator and vendor agent is a uAgent with an Agentverse mailbox; the API talks to them over the network instead of in-process	"Are agents registered on Agentverse?" is a judging question	Fetch.ai	Parikshit	P0	5
3	Gemini goal compiler live, with a 30-goal accuracy test	The compiler is the first thing judges see	Gemini	Dev	P0	4
4	Convex live state: group room and vendor portal update instantly; the API survives a restart	Live two-laptop demo; Convex track	Convex	Kevin	P0	4
5	Public deployment: API and agents on a Vultr VM, web on Vercel, HTTPS	Judges try it on their phones; Vultr track	Vultr	Kevin	P0	3

#	Feature	Why it matters	Sponsor	Owner	Priority	Hours
6	"Why this plan works for you": each person's agent scores the final plan against their own private preferences, and Gemini writes a two-sentence explanation	Makes the privacy-preserving negotiation understandable	Gemini	Dev	P1	3
7	Moss ranking for matches, plus natural-language vendor search ("boat dive with gear near Carmel")	Visible semantic search with scores	Moss	Dev	P1	3
8	Group-size nudge: the vendor's agent tells the mediator "one more person drops everyone to the next price tier", and the room offers to invite the next match	A real agent-to-agent offer with a visible saving	Fetch.ai	Parikshit	P1	3
9	x402 testnet deposits at commit, with a transaction link	Real payment rail, clearly testnet	Coinbase	Kevin	P1	4
10	Apify listed prices: an Actor scrapes public vendor price pages into the catalog as "Listed price, not a quote"	Real-world data next to agent quotes	Apify	Dev	P1	3
11	Auto-replace a member: when someone leaves, the mediator invites the next eligible person and renegotiates	Shows resilience live on stage	Fetch.ai	Parikshit	P1	2
12	Negotiation replay: a scrubber that replays rounds v1 to v4 with the diff of each step	The clearest way to show judges the agents worked		Kevin	P1	3
13	Vendor reply suggestions: Gemini drafts a reply to the group in the vendor's language, which staff can edit, speak or send	Strengthens the vendor side and voice features	Gemini	Dev	P2	2

#	Feature	Why it matters	Sponsor	Owner	Priority	Hours
14	Payment Protocol on ASI:One: a small Rally booking fee paid through Fetch.ai's protocol	Fetch.ai's monetization question	Fetch.ai	Parikshit	P2	3
15	Calendar invite (.ics) and shareable plan card on the confirmation page	Polish people notice		Kevin	P2	1

Cut order if time runs short: 15, 13, 14, 10, 12, 9. Never cut 1 to 5.

Work split

Each person owns one layer and one set of folders, so the three branches rarely touch the same file. Kevin also owns integration and merges.

Person	Owns	Branch	Folders they edit	Features
Parikshit	Agents and negotiation (Fetch.ai)	`feat/agents-network`	`agents/`, `services/api/rally/negotiation/`	1, 2, 8, 11, 14
Dev	AI, search and data (Gemini, Moss, Apify)	`feat/ai-search-data`	`services/api/rally/compiler.py`, `matching.py`, new `services/api/rally/ai/`, `data/`, `tests/evaluation/`	3, 6, 7, 10, 13
Kevin	Platform, frontend, payments and demo (Convex, x402, Vultr)	`feat/platform-live`	`convex/`, `apps/web/`, `services/api/rally/store.py`, `main.py`, new `infra/`, `README.md`	4, 5, 9, 12, 15 and the demo

Files more than one person needs:

- `services/api/rally/service.py`: everyone adds to it, so each person adds their new methods in a clearly marked block at the end of the class and keeps edits elsewhere to a few lines.
- `packages/contracts/`: changes go through Kevin (see Shared contracts).
- `apps/web/`: Dev and Parikshit add backend fields; Kevin builds the screens. Anyone may add a small component under `apps/web/components/rally/` if it is new.

Personal work docs, with each person's tasks, files, tests, handoffs and done-checks:

- Kevin: Kevin: platform, frontend and demo work doc
- Parikshit: Parikshit: agents and Fetch.ai work doc
- Dev: Dev: AI, search and data work doc

Shared contracts and merge rules

Kevin merges one small contracts PR into `main` first, before 4 pm, so all three branches start from the same message and field names. After that, a contract change is a separate small PR that Kevin merges within 30 minutes.

Fields and endpoints agreed now:

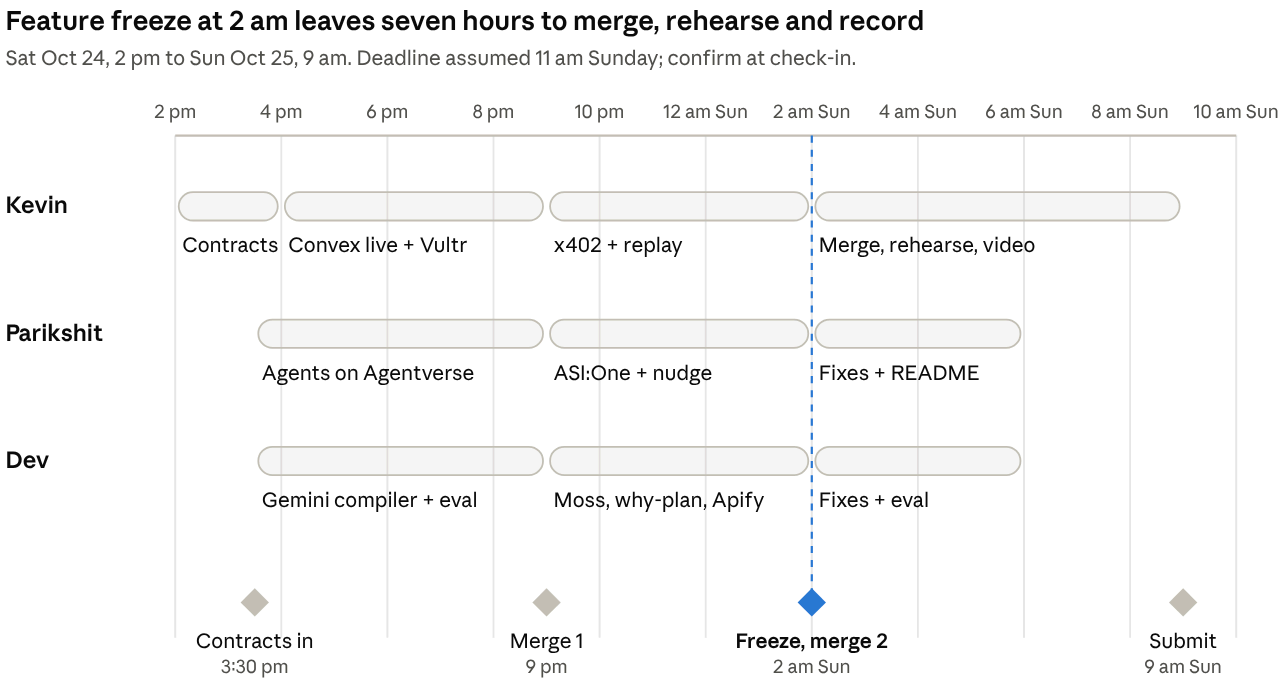
Change	File	Used by	Added by
`Deal.nudge`: `{providerId, extraPeople, newPricePerPerson, savingPerPerson}` or null	`packages/contracts/rally_contracts/deal.py`	Parikshit writes, Kevin shows	Kevin (contracts PR)
`Deal.replacements`: list of `{leftUserId, invitedUserId, at}`	`deal.py`	Parikshit writes, Kevin shows	Kevin (contracts PR)
`GET /v1/deals/{id}/explanation`: the caller's own "why this plan" text; never in the shared snapshot	`services/api/rally/main.py`	Dev writes, Kevin shows	Dev
`Quote.provenance = "cached"` for Apify listed prices (already exists)	`deal.py`	Dev writes, Kevin shows	none
`Commitment.method = "x402_testnet"` and `txRef` (already exist)	`commitment.py`	Kevin	none
New env flags: `TRANSPORT=agentverse`, `RANKER=moss`, `COMPILER=gemini`, `PROVIDERS=seeded+apify`, `PAYMENTS=x402_testnet`, `STORE=convex`	`.env.example`, `services/api/rally/core.py`	everyone	each owner adds their own line

Rules:

1. Every new integration sits behind its flag, and the default stays the working fallback. `main` must always run offline with `python scripts/rally.py dev`.
2. `python scripts/rally.py test` passes before every PR. Add at least one test per feature.
3. After changing contracts, run `python scripts/rally.py contracts` and commit the regenerated `packages/contracts/ts/index.ts`.

4. Rebase on `main` after each merge checkpoint. Small PRs, one feature each, titled `[agents]`, `[ai]` or `[platform]`.
5. Don't reformat files you don't own. Secrets go in `.env` only, never in commits.

Timeline and merge order



team timeline · 3 lanes, 4 gates

Work runs in three parallel lanes with four gates; nothing new starts after the 2 am freeze. Merge order at each gate: Kevin's platform branch first (it carries contracts and the store), then Parikshit's agents, then Dev's AI and data, each rebased on the previous merge, with `python scripts/rally.py test` and the two-sided browser test run after every merge.

Demo script and submission

The 3-minute demo starts in ASI:One and ends with a vendor confirming a booking, so every sponsor appears on screen in one continuous story. Three laptops: Kevin drives the customer side, Dev plays the vendor, Parikshit narrates the agent conversation.

1. 0:00–0:20. The problem: a group chat that never turns into a booking.

2. 0:20–0:50. In ASI:One, type "Scuba in Monterey next month under $300, I'm Open Water." The Rally agent replies with the compiled goal (Gemini) and 4 matched divers (Moss), with one person rejected and the reason.
3. 0:50–1:40. Open the group room link from that reply. Agents negotiate v1 to v4 live (Convex). Parikshit points to the agent conversation: Kai's agent rejects shore dives, Ravi's agent asks for Point Lobos. Each person's "why this plan" shows. The vendor's agent offers the group-size nudge.
4. 1:40–2:20. Vendor laptop: Marcus sees the drafted quote, gives a small discount, and sends a Spanish voice message that the group reads translated.
5. 2:20–2:45. Members approve; deposits settle on Base Sepolia (x402) with a transaction link; Marcus confirms the booking.
6. 2:45–3:00. Close on the measured numbers: 0 hard-constraint violations in 50 test negotiations, and the Agentverse agent count.

Fallback: if the venue network fails, run `python scripts/rally.py dev` offline (template compiler, local ranking, in-process agents, simulated ledger) and play the recorded video for the ASI:One part.

Submission checklist:

- [ ] Rules and deadline confirmed with each sponsor booth (Kevin)
- [ ] Public GitHub repo, `main` green, README with architecture, sponsor usage, agent names and addresses, and both Innovation Lab badges (Kevin, Parikshit)
- [ ] ASI:One chat session URL and Agentverse agent URLs (Parikshit)
- [ ] Public URL on Vultr + Vercel works from a phone (Kevin)
- [ ] Compiler accuracy and negotiation numbers committed in `tests/evaluation/results/` (Dev, Parikshit)
- [ ] 3 to 5 minute demo video on Devpost (Kevin)
- [ ] Three full rehearsals from `python scripts/rally.py reset`, each under 3 minutes (all)
- [ ] Devpost submitted at least 2 hours before the deadline (Kevin)